

Projet Xplain

Manuel utilisateur

Versions

28/12/2024 **1.0**

Création du document et rédaction des points majeures

Table des matières

Table des matières.....	2
Introduction.....	3
Manuel d'utilisation.....	4
Le produit en lui même.....	4
L'application.....	4
Le projet.....	4
Préparer sa machine.....	6
Monter l'application.....	7
Lancer l'application.....	7
Détails des cas d'utilisation.....	7
Visualiser la liste des conversations passées.....	8
Créer une nouvelle conversation.....	9
Corriger une classe Java.....	10
Visualiser les détails d'un message envoyé.....	13
Choisir le modèle d'IA de correction.....	14
Reprendre une conversation.....	15
Supprimer une conversation.....	16
Rechercher une conversation.....	17
Éditer le nom d'une conversation existante.....	18
Éditer un message précédemment envoyé.....	20
Conclusion.....	21

Introduction

Bonjour et bienvenue dans ce manuel dédié à l'utilisateur. Nous exposerons ici tout ce qui vous permettra de comprendre et utiliser notre projet . Nous ne rentrerons pas dans les détails techniques, le développement même, le choix des technologies et autres problèmes et solutions liés au développement. Pour cela, veuillez vous référer à la documentation technique présente avec le document présent. Nous passons maintenant au résumé du besoin, nous vous souhaitons une agréable lecture.

Manuel d'utilisation

Ce manuel présente tout d'abord l'application produite, puis les détails de son utilisation en commençant par sa mise en place puis en finissant par les cas d'utilisation spécifiquement détaillés.

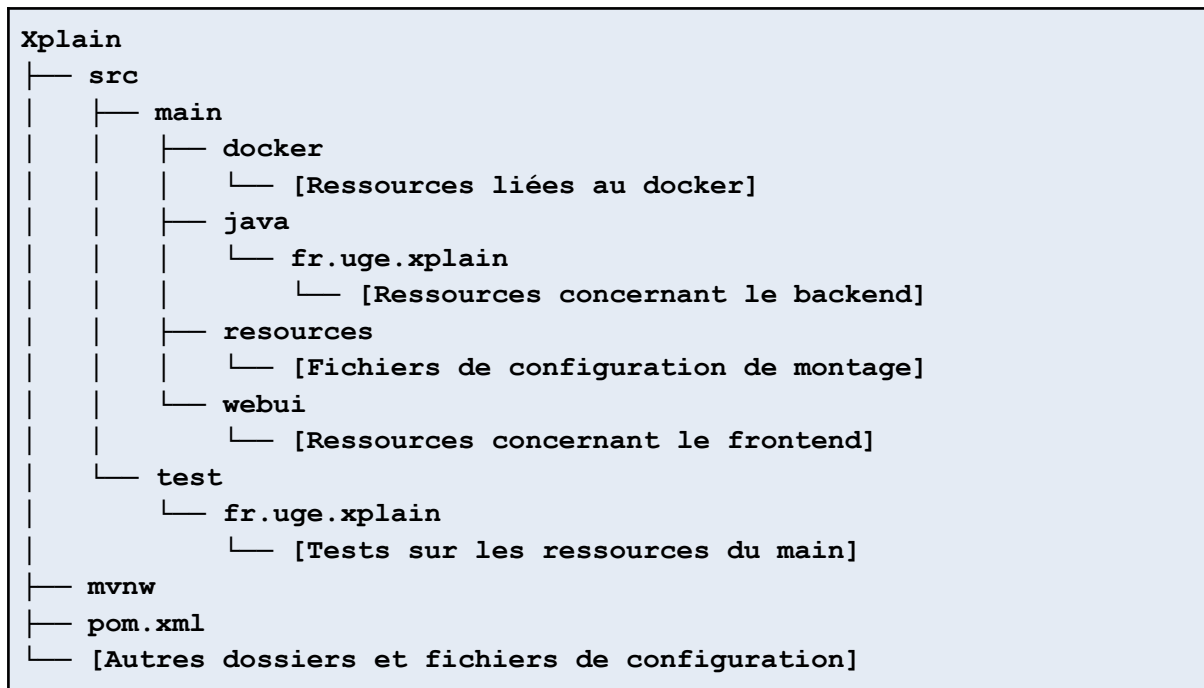
Le produit en lui même

L'application

Le projet [Xplain](#) consiste en la création d'une application web d'aide au débogage de classes Java spécifiquement. Elle reproduit l'interface classique de discussion avec des LLM (Large Language Model) modernes avec un historique de conversations, où chaque conversation consiste en une série d'interactions entre l'utilisateur et Marx, un modèle de LLM parmi ceux présents dans une liste. Un message de l'utilisateur doit contenir une classe Java, une première réponse sera celle du compilateur qui affichera les messages de compilation, ceux ci seront suivis par la correction "intelligente" du LLM courant.

Le projet

L'archive une fois décompressée présente de nombreux répertoires et fichiers. Vous devriez voir l'arborescence suivante:



Voici une rapide explication de cette arborescence, vous pouvez la passer si vous ne vous intéressez pas à la maintenance du code.

- **src**: Regroupe tous les fichiers sources de l'application, vous y retrouverez notamment :
 - **java**: Contient le package principal contenant les sous packages et donc les classes assurant le backend de l'application
 - **webui**: Contient les fichiers JavaScript, HTML et CSS pour le frontend de l'application (voir documentation [Quinoa](#), [Vite](#), [SolidJS](#), [Bulma](#))
- **test**: Contient les fichiers de tests pour les sources contenu dans le main (voir documentation [JUnit5](#))
- **mvnw**: Fichier contenant le script shell de compilation de l'application (on l'utilisera à la place de la commande `mvn` habituelle)
- **pom.xml**: Fichier contenant toutes les dépendances, plugins et profils du projet [Maven](#). Utile pour trouver tous les frameworks utilisés et leur version.

Préparer sa machine

Cinq conditions doivent être réunies pour pouvoir lancer notre projet (en plus des triviales comme “posséder un ordinateur”):

- Disposer de [GaalVM](#), version 23+, sur sa machine
- Disposer de [Maven](#), version 3.9+, sur sa machine
- Disposer de [NodeJs](#) (22+) et de la commande npm (11+)
- Être en mesure de lancer des commandes depuis une invite de commandes (Windows) ou un terminal (Linux)
- Disposer d’un système d’exploitation Linux x86_64, MacOS x86_64 ou Windows x86_64.

Afin de passer à l’étape suivante, il vous faut ouvrir votre logiciel lanceur de commandes et vous placer dans le répertoire **Xplain** de l’application. Normalement, vous devriez voir, en lançant la commande `ls`, au moins

```
mvnw    pom.xml    src
```

La commande `java -version` devrait afficher quelque chose similaire à

```
java version "23.0.1" 2024-10-15
Java(TM) SE Runtime Environment Oracle GraalVM 23.0.1+11.1 (build
23.0.1+11-jvmci-b01)
Java HotSpot(TM) 64-Bit Server VM Oracle GraalVM 23.0.1+11.1
(build 23.0.1+11-jvmci-b01, mixed mode, sharing)
```

Et la commande `mvn -v` devrait afficher quelque chose comme

```
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)
Maven home:
C:\your\maven\home\apache-maven-3.9.9-bin\apache-maven-3.9.9
```

Monter l'application

Monter l'application se fait à la condition d'avoir respecté la préparation indiquée dans la partie précédente. Pour monter l'application, il vous suffit de lancer la commande suivante:

```
./mvnw package
```

N'hésitez pas à lire la documentation [Maven](#) pour connaître les autres options qui peuvent être ajoutées à la précédente commande.

Lancer l'application

Si le montage s'est produit sans problème, vous devriez pouvoir lancer le serveur Quarkus, et donc l'application, avec la commande suivante:

```
java --add-modules jdk.incubator.vector  
--enable-native-access=ALL-UNNAMED -jar target/Xplain-runner.jar
```

Vous êtes désormais en mesure de vous connecter au serveur en ouvrant votre navigateur et allant à l'url <http://0.0.0.0:8080>. Vous pouvez désormais naviguer dans l'application et tester les différents cas d'utilisation listés ci-dessous, ils sont triés par ordre d'importance.

Détails des cas d'utilisation

Nous détaillons dans cette partie chaque cas d'utilisation de notre application. Un cas d'utilisation est ici décrit par un intitulé, une rapide description des raisons de son existence, les préconditions à remplir afin de pouvoir effectuer l'utilisation, les postconditions (ou "état final attendu"), le niveau d'importance associé à ce cas d'utilisation accompagné d'une estimation de sa fréquence d'occurrence, et enfin les scénarios d'utilisation. Le scénario nominal décrit étape par étape le comportement normal des différents acteurs, un alternatif décrit les

variantes prévues pour ce cas tandis que les exceptionnels sont des remparts face à d'éventuelles mauvaises pratiques.

UC #01

Visualiser la liste des conversations passées

L'utilisateur voit sous forme d'historique toutes les autres conversations qu'il a eu avec les modèles.

Précondition(s) Avoir lancé l'application

Postcondition(s) La liste de toutes les conversations est accessible par ordre antichronologique.

Importance : Haute ▾

Fréquence : Haute ▾

Scénario nominal

Etape	Utilisateur	Xplain
1		Présente la liste des conversations passées par ordre antichronologique.
Fin du cas d'utilisation		

Scénario alternatif: Retour à l'historique depuis une conversation

Etape	Utilisateur	Xplain
1.1	Demande le retour à l'historique	
2.1		Quitte la conversation courante
Saut à l'étape 1		
Fin du cas d'utilisation		

UC #02

Créer une nouvelle conversation

L'utilisateur doit pouvoir créer une nouvelle conversation afin de pouvoir interagir avec un modèle d'IA par la suite.

Précondition(s)

Avoir accès à l'historique

Postcondition(s)

Une nouvelle conversation vierge est créée. Elle est immédiatement ajoutée au système et ensuite présentée à l'utilisateur.

Importance : Haute ▾

Fréquence : Haute ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Demande la création d'une nouvelle discussion	
2		Crée une nouvelle discussion avec un nom et une description par défaut
3		L'ajoute à l'historique
4		Présente la nouvelle discussion
Fin du cas d'utilisation		

UC #03

Corriger une classe Java

Une conversation seule ne fait rien, l'utilisateur peut l'alimenter en envoyant en un "message" sa classe Java qu'il souhaite voir se faire corriger.

Précondition(s) Être dans une discussion

Postcondition(s) Les messages de compilation de la classe sont présentés et le modèle d'IA courant corrige les problèmes de la classe s'il y en a

Importance : Haute ▾

Fréquence : Haute ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Donne le code de la classe qu'il souhaite corriger	
2		Nomme la discussion selon le nom de la classe
3		Indique les messages de compilation de la classe
4		Corrige la classe en fonction de ces messages
5		Ajoute une description à la discussion en fonction des 3 premières lignes formulée par le modèle d'IA.
Fin du cas d'utilisation		

Scénario alternatif: Classe sans bugs

Etape	Utilisateur	Xplain
3.1		N'indique aucun message d'erreur dans la compilation
4.1		Indique que la classe est sans bug et peut être utilisée comme tel
5.1		Apporte éventuellement des optimisations à la classe
Fin du cas d'utilisation		

Scénario alternatif: Discussion déjà existante

Etape	Utilisateur	Xplain
2.2		Indique les messages de compilation de la classe
3.2		Corrige la classe en fonction de ces messages
Fin du cas d'utilisation		

Scénario exceptionnel: Conversation quittée pendant la correction

Etape	Utilisateur	Xplain
4.3	Quitte la conversation	
5.3		Stop la correction
Saut à l'étape 5		
Fin du cas d'utilisation		

Scénario exceptionnel: Modèle d'IA changé pendant la correction

Etape	Utilisateur	Xplain
4.4	Change de modèle	
5.4		Continue la correction avec le modèle avec lequel il a commencé à corriger
Saut à l'étape 5		
Fin du cas d'utilisation		

Scénario exceptionnel: Classe initiale changée pendant la correction

Etape	Utilisateur	Xplain
4.4	Change sa classe	
5.4		Continue la correction avec la classe initiale
Voir UC #09		
Saut à l'étape 5		
Fin du cas d'utilisation		

UC #04

Visualiser les détails d'un message envoyé

Un message contient plus d'informations que simplement son contenu ou son créateur, ces détails peuvent intéresser à connaître pour l'utilisateur.

Précondition(s) Être sur une page de conversation

Postcondition(s) Les détails d'un message (date et heure de création, contenu et émetteur) sont visibles

Importance : Haute ▾

Fréquence : Haute ▾

Scénario nominal

Etape	Utilisateur	Xplain
1		Indique les détails de chaque message à côté de ceux-ci
Fin du cas d'utilisation		

UC #05

Choisir le modèle d'IA de correction

Un utilisateur peut choisir le LLM qui se cache derrière le pseudo "Marx" et qui corrigera son code.

Précondition(s) Avoir créé une conversation et être sur sa page

Postcondition(s) Modèle d'IA changé

Importance : Haute ▾

Fréquence : Moyenne ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Demande à voir la liste des LLM disponibles	
2		Présente la liste des LLM accessibles depuis l'application
3	Choisit un LLM dans la liste	
4		Change le modèle
Fin du cas d'utilisation		

UC #06

Reprendre une conversation

Une conversation déjà créée peut être reprise plus tard afin de permettre un suivi de projet se déroulant sur plusieurs sessions de code.

Précondition(s)

Avoir accès à l'historique

Postcondition(s)

L'ancienne conversation est reprise et la correction suit le cours de l'UC #04.

Importance : Moyenne ▾

Fréquence : Moyenne ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Demande la visualisation d'une conversation spécifique	
2		Présente la conversation
3	Demande l'ajout d'une nouvelle classe Java	
4		Reprend la correction
Voir UC #04		
Fin du cas d'utilisation		

UC #07

Supprimer une conversation

Afin d'alléger sa base de données, l'utilisateur peut supprimer certaines conversations existantes. Cette suppression est définitive et immédiatement appliquée au système.

Précondition(s)	Visualiser l'historique de conversations et celle à supprimer
Postcondition(s)	La conversation est supprimée définitivement et donc désormais inaccessible

Importance : **Faible** ▾

Fréquence : **Moyenne** ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Demande la suppression d'une conversation	
2		Supprime la conversation du système
3		Met à jour l'historique et le présente
Fin du cas d'utilisation		

UC #08

Rechercher une conversation

L'utilisateur peut rechercher une conversation existante au travers d'un outil de recherche, particulièrement utile lorsque l'historique s'étend sur de très nombreuses conversations.

Précondition(s)

Visualiser l'historique de conversation

Postcondition(s)

La ou les conversations correspondant à la recherche sont visibles, les autres ne le sont plus

Importance : **Faible** ▾

Fréquence : **Moyenne** ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Indique la recherche d'une chaîne de caractères	
2		Filtre les conversations, ne laissant apparaître que les conversations ayant cette chaîne dans leur nom ou leur description
Fin du cas d'utilisation		

UC #09

Éditer le nom d'une conversation existante

L'utilisateur peut choisir d'éditer le nom d'une conversation à sa guise (en ne dépassant pas les 255 caractères)

Précondition(s) Visualiser l'historique de conversations et celle à éditer

Postcondition(s) Le nom est changé selon les choix de l'utilisateur

Importance : **Faible** ▾

Fréquence : **Faible** ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Demande l'édition du nom d'une conversion	
2		Indique le passage en mode édition
3	Change le nom à sa guise	
4	Valide le changement	
5		Modifie le nom de la conversation
6		Indique le nom nouvellement modifié
Fin du cas d'utilisation		

Scénario exceptionnel: Nom trop long ou vide

Etape	Utilisateur	Xplain
5.1		Empêche la validation
		Saut à l'étape 3
		Fin du cas d'utilisation

UC #10

Éditer un message précédemment envoyé

Plutôt que de réécrire son code, un utilisateur peut choisir d'éditer un message précédemment écrit par lui-même ou l'IA correctrice. Sa correction compte comme une nouvelle classe à valider.

Précondition(s)

Visualiser une conversation déjà entamée

Postcondition(s)

Un des messages précédemment ajoutés (de l'utilisateur ou de l'IA) est modifié et corrigé à nouveau

Importance : **Faible** ▾

Fréquence : **Faible** ▾

Scénario nominal

Etape	Utilisateur	Xplain
1	Demande la modification d'un message existant	
2		Indique que le mode édition est activé
3	Modifie le message selon son souhait	
4		Propose la sauvegarde du message modifié
5	Sauvegarde son message	
Voir UC#04		
Fin du cas d'utilisation		

Conclusion

Vous connaissez désormais toutes les formes d'utilisation de l'application du projet Xplain. Vous comprendrez que la correction de classes Java une par une n'est pas optimale pour l'aide à la production de projet complet. L'application Xplain a pour but implicite de supporter les développeurs novices au développement de leurs premières classes et la résolution de leurs premiers bugs.

Pour rapporter des problèmes avec l'application vous pouvez contacter les développeurs sur leurs adresses mails étudiantes¹. Nous espérons avoir été clair dans ce manuel et vous souhaitons une bonne utilisation pour vos débuts dans l'informatique.

¹ max.tekpa@edu.univ-eiffel.fr – marc.le-coquil@edu.univ-eiffel.fr